

Sui tetti di Pisa (parkour)

Tommaso si è recentemente appassionato al parkour. Sfortunatamente Pisa, la città in cui vive Tommaso, non dispone di alcuna palestra in cui potersi allenare e quindi è costretto a fare pratica sui tetti della città.



Il quartiere in cui si allena Tommaso è costituito da una fila di N case e l' i -esima ($0 \leq i < N$) è alta S_i metri. Quando si trova sul tetto i , Tommaso può fare un salto di lunghezza compresa tra A_i e B_i metri (estremi inclusi). Quindi, può raggiungere le posizioni da $i + A_i$ a $i + B_i$. Il suo obiettivo è quello di raggiungere il parco, in posizione N .

Per quanto sia un ragazzo temerario, Tommaso è vittima di una condizione abbastanza avversa alla vita da parkourista: soffre di vertigini! Quando si allena infatti, cerca sempre di rimanere sui tetti più bassi.

In particolare, partendo dal tetto 0 Tommaso vuole trovare il percorso che lo porta in posizione N che **minimizza l'altezza massima** dei tetti su cui salta. Aiutalo in questo compito!

Implementazione

Dovrai sottoporre un unico file, con estensione `.cpp`.

📎 Tra gli allegati a questo task troverai un template `parkour.cpp` con un esempio di implementazione.

Dovrai implementare la seguente funzione:

```
C++ | int salta(int N, vector<int> S, vector<int> A, vector<int> B);
```

- L'intero N rappresenta il numero di case.
- Il vettore S , indicizzato da 0 a $N - 1$, contiene le altezze in metri delle case.
- I vettori A e B , indicizzati da 0 a $N - 1$, contengono gli estremi dell'intervallo di case su cui può saltare Tommaso quando si trova sulla casa i . In altre parole, Tommaso dal tetto della casa i può saltare in posizione j con $A[i] \leq j \leq B[i]$.
- La funzione dovrà restituire la minima altezza massima dei tetti di un percorso che partendo dalla casa 0 arriva in posizione N

Il grader chiamerà la funzione `salta` e ne stamperà il valore restituito sul file di output.

Grader di prova

Nella directory relativa a questo problema è presente una versione semplificata del grader usato durante la correzione, che potete usare per testare le vostre soluzioni in locale. Il grader di esempio legge i dati da `stdin`, chiama le funzioni che dovete implementare e scrive su `stdout`, secondo il seguente formato.

Il file di input è composto da $N + 1$ righe, contenenti:

- Riga 1: l'unico intero N .
- Riga $2 + i$: i tre valori $S[i]$, $A[i]$, $B[i]$.

Il file di output è composto da un'unica riga, contenente:

- Riga 1: il valore restituito dalla funzione `salta`.

Assunzioni

- $1 \leq N \leq 1\,000\,000$.
- $0 \leq S[i] \leq 1\,000\,000$ per ogni $i = 0, \dots, N - 1$.
- $1 \leq A[i] \leq B[i] \leq N - i$ per ogni $i = 0, \dots, N - 1$.

Assegnazione del punteggio

Il tuo programma verrà testato su diversi test case raggruppati in subtask. Per ottenere il punteggio relativo ad un subtask, è necessario risolvere correttamente tutti i test che lo compongono.

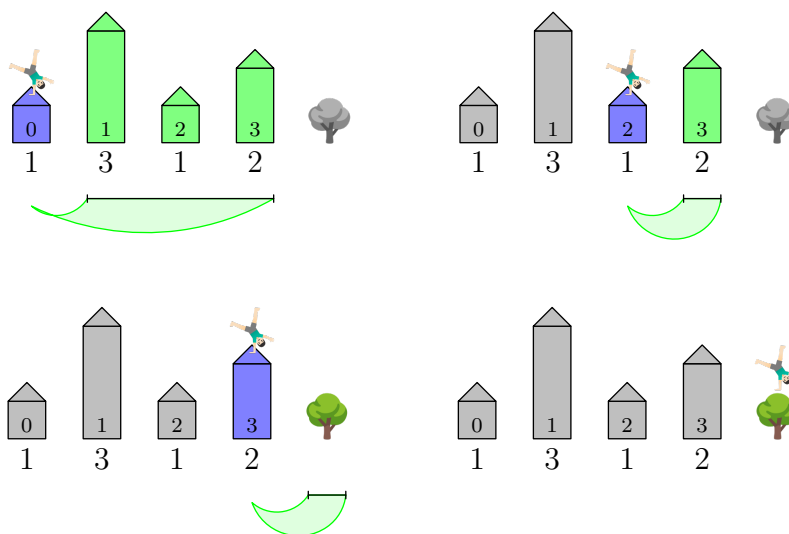
- **Subtask 1 [0 punti]**: Casi d'esempio.
- **Subtask 2 [5 punti]**: $N \leq 10$.
- **Subtask 3 [10 punti]**: $N \leq 100$ e $S[i] \leq 100$ per ogni $i = 0, \dots, N - 1$.
- **Subtask 4 [10 punti]**: $N \leq 10\,000$ e $S[i] \leq 10\,000$ per ogni $i = 0, \dots, N - 1$.
- **Subtask 5 [15 punti]**: $N \leq 10\,000$.
- **Subtask 6 [30 punti]**: $N \leq 100\,000$.
- **Subtask 7 [10 punti]**: $A[i] = 1$.
- **Subtask 8 [20 punti]**: Nessuna limitazione aggiuntiva.

Esempi di input/output

stdin	stdout
4 1 1 3 3 3 3 1 1 1 2 1 1	2
6 2 2 3 1 3 4 2 1 2 3 2 3 1 1 2 1 1 1	2

Spiegazione

Nel **primo caso di esempio** Tommaso parte dal tetto 0 e può raggiungere i tetti 1, 2 e 3. Se salta sul tetto 2, poi sul tetto 3 e infine arriva a destinazione, l'altezza massima raggiunta è 2. Questa non è l'unica soluzione, ma non è possibile raggiungere il parco saltando solo su tetti più bassi di 2.



Nel **secondo caso di esempio** con il primo salto può raggiungere i tetti 2 e 3. Se il percorso fosse 0, 2, 4, 6, l'altezza massima raggiunta è 2. Non è possibile raggiungere il parco saltando solo su tetti più bassi di 2.

